

Campus Hub – Project Report

1. Real World Problem Identification

Problem Statement

University students frequently face challenges in accessing academic resources, equipment, and materials. Common issues include:

High Cost of Resources: Textbooks, lab equipment, and electronics are expensive, often used briefly, then left unused.

Resource Waste: Students possess items they no longer need but have no easy way to share or donate them.

Limited Access: Not all students can afford essential resources, creating educational inequality.

No Centralized Platform: Existing solutions (social media groups, notice boards) are disorganized, lack verification, and have no tracking mechanism.

Trust Issues: Peer-to-peer sharing lacks accountability and proper return workflows.

Impact

Students spend unnecessarily on items they'll use temporarily.

Valuable resources sit idle instead of benefiting others.

Educational opportunities are limited by financial constraints.

Environmental waste from underutilized resources.

2. Proposed Solution

Campus Hub is a dedicated resource-sharing platform designed specifically for university students with the following features.

Core Functionalities

1. Secure Authentication System

Email-based registration with OTP verification.

JWT token authentication for secure sessions.

University email verification ensures only genuine students participate.

2. Resource Management

Upload resources with images and detailed descriptions.

Categorization: Books, Electronics, Lab Equipment, Stationery, Miscellaneous.

Two sharing types: **Share** (lend temporarily) or **Donate** (give permanently).

Edit/Delete functionality for resource owners.

3. Request and Approval Workflow

Students can request to borrow items with custom return dates.

Lenders approve or decline requests with reasons.

Status tracking: Pending → Approved/Declined.

Real-time notifications for request updates.

4. Transaction Tracking System

Dashboard for borrowed items (as borrower).

Dashboard for lent items (as lender).

Return workflow: Borrower marks returned → Lender confirms.

Dispute handling for problematic returns.

Complete transaction history.

5. Advanced Search and Discovery

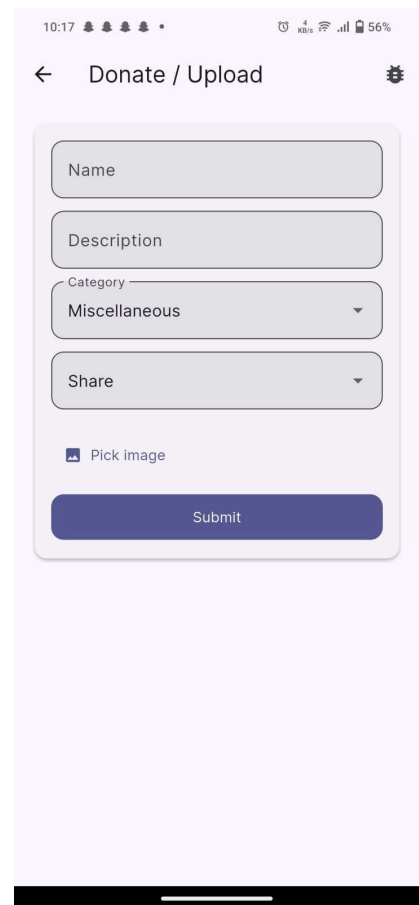
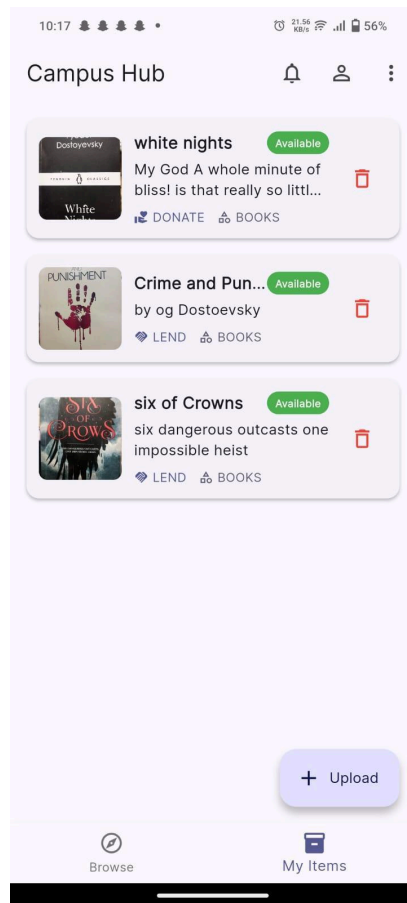
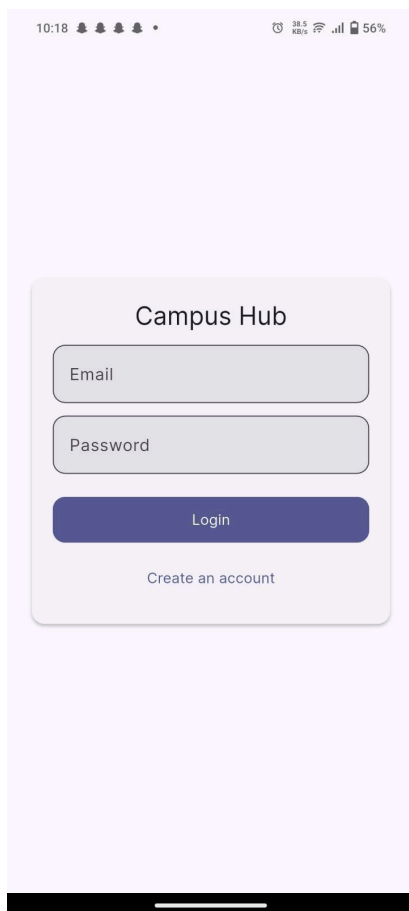
Filter by category, sharing type, and keywords.

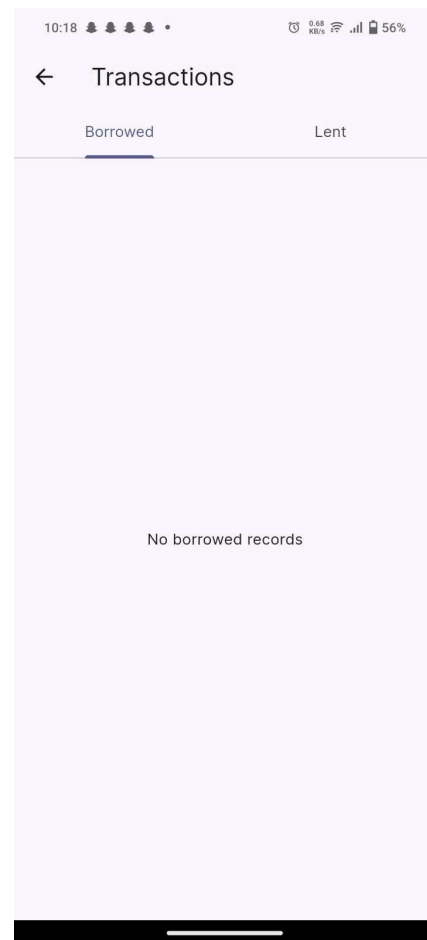
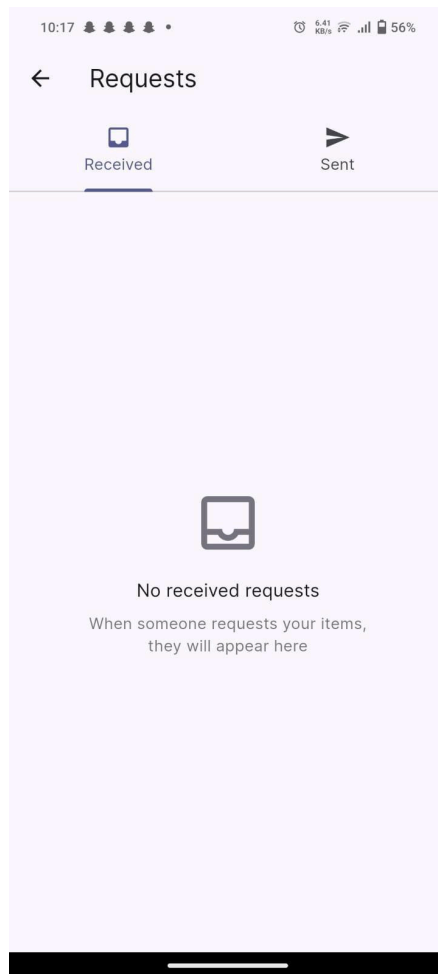
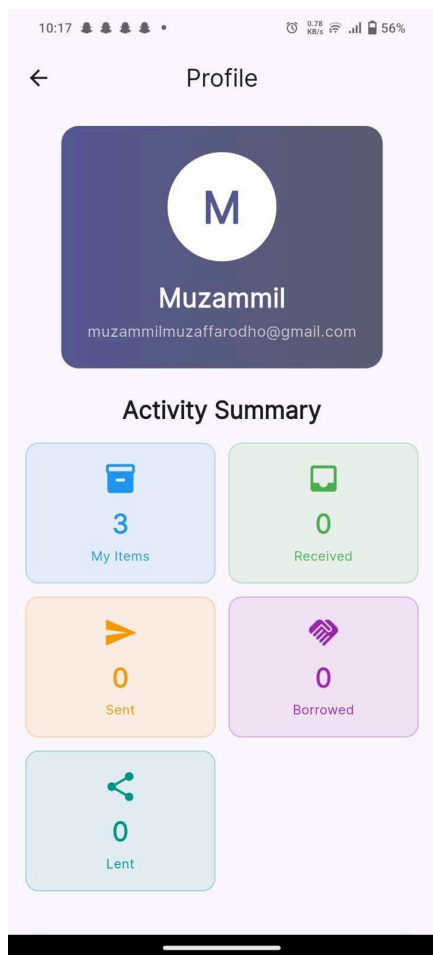
Browse all available resources.

View detailed item information with images.

See lender profiles and contact information.

3. Responsive User Interfaces





4. Data Storage

Database: PostgreSQL

Justification

1. **Relational Model:** Perfect for Users → Resources → Requests → Transactions.
2. **Data Integrity:** Cascading deletes, foreign key constraints, unique emails.
3. **Complex Queries:** Multi-table joins for dashboards and statistics.
4. **Scalability:** Indexing and connection pooling.
5. **JSONB Support:** Store flexible metadata.
6. **Full-Text Search:** Efficient search on resource descriptions.
7. **Mature Ecosystem:** Excellent Spring Boot integration, extensions, and backup tools.

Alternatives Rejected:

MongoDB: No ACID or strong relational mapping.

SQLite: Single-user limitation.

MySQL: Lacks JSONB and advanced SQL features.

5. APIs / Packages / Plug-ins Used

Flutter Packages

1. flutter_riverpod (v2.6.1)

Purpose: Reactive state management.

Justification: Type-safe, cached, auto-disposing providers.

Usage: Authentication, resources, transactions, and profile providers.

2. go_router (v14.6.2)

Purpose: Declarative navigation with deep linking.

Usage: Auth guards, nested routes, web compatibility.

3. *http* (v1.2.2)

Purpose: REST API client.

Usage: CRUD requests with JWT headers.

4. *flutter_secure_storage* (v9.2.2)

Purpose: Store JWT tokens securely.

Usage: Encrypted local storage for authentication.

5. *image_picker* (v1.1.2)

Purpose: Upload images from camera/gallery.

Usage: Resource photo upload with compression.

6. *cached_network_image* (v3.4.1)

Purpose: Image caching.

Usage: Efficiently display resource thumbnails.

7. *google_fonts* (v6.2.1)

Purpose: Modern typography using Inter font.

8. *intl* (v0.19.0)

Purpose: Date and time formatting.

9. *flutter_dotenv*

Purpose: Environment variable management.

6. Issues and Bugs Encountered and Resolved

1. CardTheme vs CardThemeData Type Mismatch

Fixed by replacing CardTheme with CardThemeData.

Lesson: Check Flutter version updates carefully.

2. CORS Errors in Flutter Web

Fixed via Spring Boot CORS configuration.

Lesson: Always configure CORS for frontend origins.

3. Android Emulator API Connection

Changed localhost → 10.0.2.2.

Lesson: Emulator network aliases matter.

4. JWT Expiration Handling

Added interceptor to redirect on 401 Unauthorized.

Lesson: Handle global errors uniformly.

5. Image Upload Size Limit

Added compression to 70% quality.

Lesson: Client-side optimization saves bandwidth.

6. Riverpod Provider Not Updating

Used `ref.invalidate()` after updates.

Lesson: Refresh providers explicitly when mutating data.

7. Null Safety Issues

Updated models to handle nullable fields.

Lesson: Always validate API responses.

8. GoRouter Redirect Loop

Fixed logic to exclude `/login` and `/signup` routes.

Lesson: Carefully design redirect conditions.

7. Conclusion

Campus Hub addresses resource scarcity and waste on campuses by enabling secure, verifiable sharing and donation.

It integrates a full-stack Flutter frontend, Spring Boot backend, and PostgreSQL database, demonstrating strong understanding of:

Cross-platform app development.

State management (Riverpod).

RESTful API integration.

Real-world resource exchange workflows.

